A CLEAR, PRACTICAL START WITH PYTHON

Python Beginner Study Notes

From your first program to collections, functions, files, and useful Python tips



What is inside

In-depth explanations for tutorial topics 1-10 and every Python tip visible in your screenshots. Examples are written for Python 3 and use small, readable programs.

Learn by reading, running, and changing the examples

Contents

Use the page numbers to jump to a topic. Type examples into a .py file and run them.

How to use these notes 3

1. Installation and your first program 3

2. Variables and data types 4

3. Operators and user input 5

4. If statements and conditions 6

5. Loops 7

6. Sequences and collections 8

7. Functions 9

8. Exception handling 10

9. File operations 11

10. String functions 12

11. List comprehensions 13

12. F-strings and the ternary expression 13

13. map, lambda, and docstrings 14

14. enumerate, any, and all 15

15. Reverse, join, zip, and slices 15

16. Swap variables and merge dictionaries 16

17. Find the most frequent value 17

18. Count the digits in an integer 18

Practice: try these before reading the answers 18

Quick reference 19

How to use these notes

Read each section once, then type the examples instead of only copying them. Change a value, predict the output, and run it. This loop builds understanding better than memorizing syntax.

A study routine

- Read the example and explain what it should do.
- Run it in a .py file using Python 3.
- Change one input or value and predict the result first.
- When something surprises you, inspect one value at a time with `print(value, type(value))`.

A program at a glance

A small program gets data, stores it, makes decisions, repeats work, packages reusable steps into functions, then communicates through output or files.



1. Installation and your first program

Python is both a programming language and the program that runs Python code. Write source in an editor or IDE, save it as a .py file, and ask the Python interpreter to execute it from top to bottom.

Set up a simple working loop

- Install a current Python 3 release from the official Python website or use the version provided by your course or computer.
- Choose an editor. A plain text editor works; an IDE adds syntax coloring, run controls, and error hints.
- Save a file such as `hello.py`. Avoid names such as `python.py` or `random.py`, which can shadow standard modules.
- Run it with your editor, or from a terminal: `python hello.py`. Some systems use `python3 hello.py`.

Your first program

```
print("Hello, Python!")
print("I am learning to write programs.")
```

Output: *Hello, Python! / I am learning to write programs.*

`print()` is a built-in function that displays text or values. Text goes inside matching single or double quotes.

Comments and indentation

```
# A comment explains code to a human
name = "Asha"
print("Welcome,", name)  # Inline comments start with #
```

A comment begins with # and is ignored by Python. Indentation is meaningful: it marks the body of a function, loop, or condition. Use consistent spaces, normally four per level.

Common first-program mistakes
Forgetting a closing quote or parenthesis, using curly quotes copied from a document, running a file from an unexpected folder, or naming your file after a module can cause confusing errors.

2. Variables and data types

A variable is a name that refers to a value. The assignment operator = stores the value on its right under the name on its left. It does not mean mathematical equality.

```
learner = "Mina"
lessons = 4
hours = 1.5
is_ready = True
print(learner, lessons, hours, is_ready)
```

Output: *Mina 4 1.5 True*

Common built-in types

- `int`: whole numbers such as `0`, `-12`, or `42`.
- `float`: numbers with a fractional part, such as `3.14`. Binary floating point is approximate for many decimal fractions.
- `str`: text such as `"hello"`. Strings are immutable sequences of characters.
- `bool`: logical values `True` and `False`. Capitalization matters.
- `NoneType`: the single value `None` means no value or no result was provided.

Names, conversion, and type checking

Names may contain letters, digits, and underscores, but cannot start with a digit. They are case-sensitive. Prefer descriptive snake_case names such as `total_price`, and do not use keywords such as `if` or `for`. Python is dynamically typed: a name can later refer to a value of another type, but each value still has a type.

```
age_text = "21"
age = int(age_text)
height = float("1.72")
label = str(age)
print(age + 1, height, label)
print(isinstance(age, int))
```

Output: *22 1.72 21 / True*

Conversion functions include `int()`, `float()`, `str()`, and `bool()`. Conversion may fail: `int("twenty")` raises `ValueError`. CAUTION|Do not confuse = and ==|= assigns a value. == compares values and produces `True` or `False`.

Mutable and immutable values

Numbers, booleans, strings, and tuples are immutable: operations create new values rather than changing them in place. Lists, dictionaries, and sets are mutable. This matters when two names refer to the same list; see Collections.

3. Operators and user input

Operators calculate, compare, or test logical conditions. User input arrives as text, so convert it before numeric work.

Arithmetic operators

- `+` adds; `-` subtracts; `*` multiplies.
- `/` divides and returns a float; `//` floor-divides; `%` gives the remainder.
- `**` raises to a power. For example `2 ** 3` is 8.

Floor division rounds down toward negative infinity. Thus `-7 // 2` is `-4`. Parentheses help make intended order clear.

Comparisons and logical operators

- Comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=`.
- Combine conditions with `and`, `or`, and `not`.
- `in` tests membership. `is` tests object identity; use it for `None`, not ordinary value comparison.

Read, convert, calculate, print

```
name = input("Your name: ")
minutes_text = input("Minutes studied: ")
minutes = int(minutes_text)
hours = minutes / 60
print(f"{name}, that is {hours:.2f} hours.")
```

`input()` displays a prompt and returns a string. `int()` converts the count, and the f-string formats the answer to two decimals.

A simple calculator and augmented assignment

```
first = float(input("First number: "))
second = float(input("Second number: "))
total = first + second
print(f"The sum is {total:g}")
```

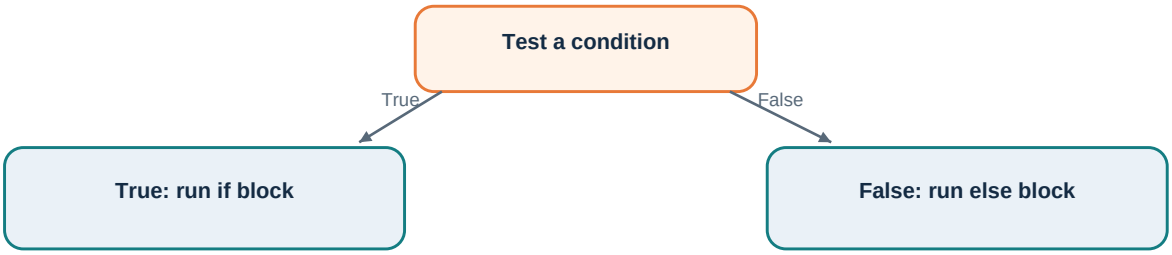
```
score = 10
score += 5   # same as score = score + 5
score *= 2   # same as score = score * 2
print(score)
```

Output: *30*

> **Invalid input**
> If someone types a word when the program expects a number, conversion raises an error. Exception handling later shows how to respond clearly.

4. If statements and conditions

A condition evaluates to a truth value. An `if` statement chooses which indented block runs. Conditions are checked from top to bottom; after one branch is selected, the others are skipped.



if, elif, and else

```
temperature = 18

if temperature >= 30:
    message = "hot"
elif temperature >= 20:
    message = "warm"
else:
    message = "cool"

print(message)
```

Output: *cool*

Use `elif` for another condition and `else` for the fallback. The colon and indentation are required.

Combining conditions and truthiness

```
age = 19
has_ticket = True
if age >= 18 and has_ticket:
    print("Entry allowed")
else:
    print("Entry not allowed")
```

`and` and `or` short-circuit: Python can stop once the result is known. In conditions, `False`, `None`, numeric zero, and empty strings or collections are false; most other values are true.

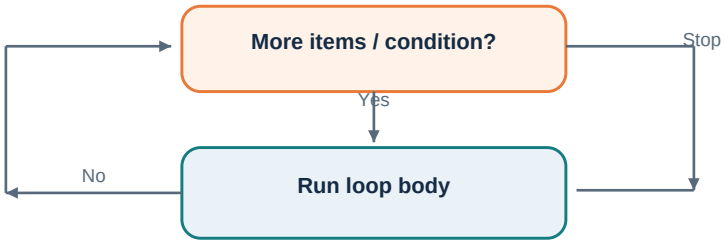
```
name = input("Name (Enter to skip): ")
if name:
    print(f"Hello, {name}!")
else:
    print("No name was entered.")
```

Common mistakes: use == to compare, not =; Python uses `and`/`or`, not &&/||. A range can read `0 <= score <= 100`.

> **Readable branches**
> State the rule plainly, for example `if balance >= price:`. Keep branches clear and avoid unnecessary nesting.

5. Loops

A loop repeats code. Use `for` to process items in an iterable such as a list or string. Use `while` when repetition depends on a condition that changes.



for loops and range

```
for name in ["Asha", "Ben", "Chen"]:
    print(f"Hello, {name}!")

for number in range(1, 5):
    print(number)
```

Output: *The names are greeted; numbers are 1, 2, 3, 4.*

`range(start, stop, step)` produces integers. The stop is excluded, so `range(1, 5)` includes 1 through 4.

while, break, continue, and loop else

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Go!")
```

Output: *3 / 2 / 1 / Go!*

A `while` loop needs an exit condition that can become false. If no state changes, it may run forever.

```
for number in range(2, 10):
    if number % 2 == 0:
        continue
    if number > 7:
        break
    print(number)
else:
    print("Finished without break")
```

`continue` skips to the next iteration. `break` exits the nearest loop. A loop's optional `else` runs only when it finishes without `break`.

> **Choosing a loop**
> Use `for` for each item or a fixed range. Use `while` when repeating until a condition changes.

> **Do not mutate a list while iterating**
> Removing values from the same list as you iterate can skip elements. Build a filtered list or iterate over a copy.

6. Sequences and collections

Collections hold multiple values. Choose one whose behavior fits the data and the operations you need.

List, tuple, set, dictionary

- A `list` is ordered and mutable: `[3, 5, 8]`. It preserves duplicates.
- A `tuple` is ordered and immutable: `(3, 5, 8)`. It suits fixed grouped values.
- A `set` stores unique hashable values and has no positional order. It is useful for membership and deduplication.
- A `dict` maps unique hashable keys to values. Dictionaries preserve insertion order in modern Python.

List indexing and methods

```
colors = ["red", "green", "blue"]
print(colors[0])       # first item
print(colors[-1])      # last item
colors.append("gold")
colors[1] = "lime"
print(len(colors), "lime" in colors)
```

Output: *red / blue / 4 True*

Indexes begin at zero; negative indexes count from the end. An invalid index raises `IndexError`.

- `append(x)` adds one item; `extend(items)` adds each item from an iterable.
- `insert(i, x)` inserts; `remove(x)` removes the first match; `pop(i)` removes and returns an item.
- `sort()` sorts in place and returns `None`; `sorted(items)` returns a new sorted list.

Tuples, sets, and dictionaries

```
point = (4, 7)
tags = {"python", "beginner", "python"}
student = {"name": "Mina", "score": 92}
print(point[0])
print(tags)
print(student["name"], student.get("score"))
```

A set removes duplicates and supports union (`|`) and intersection (`&`). A dictionary's `get()` is useful when a key may be missing. Use `items()` to iterate over key-value pairs.

```
student["grade"] = "A"
for key, value in student.items():
    print(key, value)
```

Mutability, aliases, and copies

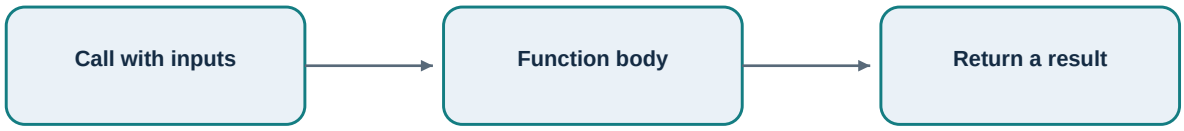
```
original = [1, 2]
alias = original
alias.append(3)
print(original)  # both names refer to the same list

copy_of_list = original.copy()
copy_of_list.append(4)
print(original, copy_of_list)
```

Assignment does not copy a mutable collection. `copy()` makes a shallow copy: the outer list is new, but nested lists are still shared. Choose by the question: order and duplicates suggests list; uniqueness suggests set; lookup by label suggests dict.

7. Functions

A function is reusable code. It can receive inputs (parameters), do work, and send a result to its caller with `return`.



The caller can store or use the returned value

Define and call

```
def greet(name):
    """Return a friendly greeting for one person."""
    return f"Hello, {name}!"

message = greet("Asha")
print(message)
```

Output: *Hello, Asha!*

`def` begins a definition. The indented body runs when called. `return` gives a value back; without it, a function returns `None`.

Parameters, arguments, and defaults

```
def rectangle_area(width, height=1):
    return width * height

print(rectangle_area(5, 3))
print(rectangle_area(width=4))
```

Output: *15 / 4*

Positional arguments match by order; keyword arguments match by name. Parameters with defaults follow required parameters.

Avoid mutable defaults

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Default expressions are evaluated once when a function is defined. A default such as `items=[]` would be shared by later calls. Use `None` and create a new list inside.

Scope and function design

A name assigned inside a function is normally local. Functions can read outer values, but hidden global state makes code harder to reuse. Prefer passing needed values as parameters.

- Give a function one clear job and a name that explains the action or result.
- Return values for later use. Print only when displaying output is part of the function's job.
- Keep inputs and outputs predictable; avoid changing globals as a side effect.

> **print is not return**
> `print(value)` displays a value. `return value` gives it back to the calling code.

8. Exception handling

An exception is raised when an operation cannot continue normally. Handling a specific expected error lets a program recover, explain the problem, or choose a safe fallback.

try, except, else, finally

```
text = input("Enter a whole number: ")
try:
    number = int(text)
except ValueError:
    print("Please enter digits such as 12 or -3.")
else:
    print(f"Twice the number is {number * 2}.")
finally:
    print("Input step finished.")
```

Python runs `try`. If a matching error occurs, it runs `except`. Optional `else` runs after success; `finally` runs either way and often handles cleanup.

Common exception types

- `ValueError`: a value cannot be converted or accepted, such as `int("abc")`.
- `TypeError`: an operation is used with unsuitable types, such as adding text and an integer.
- `IndexError`: a sequence index is outside its range.
- `KeyError`: a missing dictionary key is read with brackets.
- `FileNotFoundError`: a requested path does not exist.

Retry invalid numeric input

```
while True:
    raw = input("Positive number: ")
    try:
        value = float(raw)
    except ValueError:
        print("That was not a number.")
        continue
    if value > 0:
        break
    print("The number must be above zero.")
print("Accepted:", value)
```

Raise an exception for an invalid argument

```
def percentage(value):
    if not 0 <= value <= 100:
        raise ValueError("value must be between 0 and 100")
    return value / 100
```

Use `raise` when a function cannot produce a meaningful result. Catch errors where the program can make a useful decision. Avoid a bare `except:`; it can hide programming mistakes.

> **Read the traceback**
> A traceback shows the path through function calls and ends where the exception surfaced. Read its type and message, then inspect the named line.

9. File operations

Files keep information after a program stops. `open()` returns a file object. Prefer `with` so it closes automatically, even if an error happens.

Write and read a text file

```
with open("study_log.txt", "w", encoding="utf-8") as file:
    file.write("Python practice: 30 minutes\n")
    file.write("Topic: loops\n")

with open("study_log.txt", "r", encoding="utf-8") as file:
    contents = file.read()

print(contents)
```

A relative path is resolved from the current working directory, which may differ from the folder holding the script. An absolute path starts from the file system root.

File modes

- `r` reads; the file must exist.
- `w` writes from the start and creates or truncates a file.
- `a` appends at the end and creates a file if needed.
- `x` creates a new file and fails if it already exists.
- Add `b` for binary data such as image bytes.

Append and read line by line

```
from pathlib import Path

log_path = Path("study_log.txt")
with log_path.open("a", encoding="utf-8") as file:
    file.write("Topic: files\n")

with log_path.open("r", encoding="utf-8") as file:
    for line in file:
        print(line.rstrip())
```

Line-by-line iteration works well for large text files. `rstrip()` removes line endings and other trailing whitespace without removing leading spaces.

Handle a missing file

```
from pathlib import Path

path = Path("notes.txt")
try:
    text = path.read_text(encoding="utf-8")
except FileNotFoundError:
    text = ""
    print("No notes file yet; starting empty.")
```

Protect important data
Mode `w` replaces old contents immediately. Use `a` to add to a log, or `x` when overwriting must be prevented.

10. String functions

A string is immutable text. Methods return new strings rather than changing the original. Strings are sequences, so they support indexing, slicing, iteration, and membership.

Useful string methods

- `lower()` and `upper()` change letter case.
- `strip()` removes whitespace at both ends; `lstrip()` and `rstrip()` remove from one side.
- `replace(old, new)` returns text with matching pieces replaced.
- `split(separator)` returns a list; without a separator it groups runs of whitespace.
- `find(part)` returns the first index or -1. Use `in` for a yes/no check.

```
raw = "  Learn Python today  "
clean = raw.strip()
words = clean.split()
fixed = clean.replace("today", "step by step")
print(clean.lower())
print(words)
print(fixed)
```

Output: *learn python today / ['Learn', 'Python', 'today'] / Learn Python step by step*

Index and slice text

```
word = "Python"
print(word[0], word[-1])
print(word[1:4])
print(word[:2], word[2:])
```

Output: *P n / yth / Py thon*

A slice includes its start and excludes its stop. Missing start or stop means beginning or end. Strings are immutable, so assigning `word[0]` is an error; build new text instead.

Normalize simple user input

```
answer = input("Continue? ").strip().lower()
if answer in {"y", "yes"}:
    print("Continuing")
elif answer in {"n", "no"}:
    print("Stopping")
else:
    print("Please enter yes or no")
```

11. List comprehensions

A list comprehension builds a new list by transforming each item and optionally keeping only items that pass a condition.

Transform and filter

```
numbers = [1, 2, 3, 4, 5, 6]
squares = [n ** 2 for n in numbers]
evens = [n for n in numbers if n % 2 == 0]
print(squares)
print(evens)
```

Output: *[1, 4, 9, 16, 25, 36] / [2, 4, 6]*

Read `[n ** 2 for n in numbers]` as "make a list of n squared for each n in numbers." The optional `if` filters items.

```
names = ["asha", "ben", "chen", "diya"]
long_names = [name.title() for name in names if len(name) >= 4]
print(long_names)
```

Output: *['Asha', 'Chen', 'Diya']*

> **Keep it readable**
> Avoid comprehensions for side effects such as `[print(x) for x in items]`. Use a regular loop. If a comprehension has many nested parts, a loop is clearer.

12. F-strings and the ternary expression

F-strings: readable formatting

An f-string starts with `f` before its quotes. Expressions inside braces are evaluated and inserted into the text.

```
name = "Mina"
score = 93.456
print(f"{name} scored {score:.1f}%")
print(f"Next target: {score + 5:.0f}")
```

Output: *Mina scored 93.5% / Next target: 98*

- `:.2f` shows a float with two decimal places.
- `:04d` pads an integer to four digits with zeroes.
- `:,` adds separators to a large number.
- `!r` shows a representation useful for inspecting spaces and quotes.

```
count = 7
print(f"{count = }")
```

Output: *count = 7*

Ternary conditional expression

The conditional expression selects one of two values: `value_if_true if condition else value_if_false`.

```
score = 74
result = "pass" if score >= 50 else "try again"
print(result)
```

Output: *pass*

Use it for a short choice. For multiple branches or multi-step actions, a normal `if`/`elif`/`else` is clearer.

13. map, lambda, and docstrings

map(): apply a function to each item

`map(function, iterable)` lazily produces one result per input and returns an iterator. Use `list()` if you need all results immediately.

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda n: n * n, numbers))
print(squares)
```

Output: *[1, 4, 9, 16]*

A named function can also be passed to `map()`. A list comprehension, `[n * n for n in numbers]`, is often more direct.

lambda: a small anonymous function

A `lambda` makes a small function containing one expression. It may have multiple parameters but cannot contain normal multi-line statements.

```
students = [("Asha", 88), ("Ben", 91), ("Chen", 79)]
by_score = sorted(students, key=lambda student: student[1])
print(by_score)
```

Output: *[('Chen', 79), ('Asha', 88), ('Ben', 91)]*

Use a named `def` when the logic is substantial or reused.

Docstrings: document functions

A docstring is a string literal at the start of a module, function, or class. Python stores it as the object's `__doc__` text.

```
def average(values):
    """Return the mean of a non-empty sequence."""
    if not values:
        raise ValueError("values must not be empty")
    return sum(values) / len(values)

print(average([2, 4, 6]))
print(average.__doc__)
```

Output: *4.0 / Return the mean of a non-empty sequence.*

> **A useful docstring**
> Explain purpose, input requirements, result, and non-obvious behavior.

14. enumerate, any, and all

enumerate(): items with an index

Use `enumerate(iterable)` when a loop needs both position and item. It starts at zero unless you set `start=`.

```
topics = ["variables", "loops", "functions"]
for index, topic in enumerate(topics, start=1):
    print(f"{index}. {topic}")
```

Output: *1. variables / 2. loops / 3. functions*

This is generally clearer than manually maintaining a counter or indexing with `range(len(topics))`.

any() and all(): test many conditions

`any(iterable)` is true when at least one item is truthy. `all(iterable)` is true when every item is truthy. They can stop as soon as the answer is known.

```
scores = [82, 91, 48]
print(any(score >= 90 for score in scores))
print(all(score >= 50 for score in scores))
print(all(score >= 0 for score in []))
```

Output: *True / False / True*

`all([])` is true because there is no counterexample in an empty collection. If empty input is invalid for your task, check that it is non-empty separately.

```
answers = ["yes", "yes", "no"]
all_yes = all(answer == "yes" for answer in answers)
has_no = any(answer == "no" for answer in answers)
print(all_yes, has_no)
```

Output: *False True*

The expression `score >= 50 for score in scores` is a generator expression; it produces results one at a time.

15. Reverse, join, zip, and slices

Reverse a list

```
letters = ["a", "b", "c"]
backwards = letters[::-1]
print(backwards)
print(list(reversed(letters)))
letters.reverse()  # changes the original
print(letters)
```

Output: *['c', 'b', 'a'] / ['c', 'b', 'a'] / ['c', 'b', 'a']*

`reversed()` returns an iterator. Slice step `-1` makes a reversed copy. `list.reverse()` changes the list and returns `None`.

join(): combine strings

The separator string calls `join()` on an iterable of strings. Each item must be a string.

```
words = ["Python", "is", "readable"]
sentence = " ".join(words)
csv_line = ", ".join(["red", "green", "blue"])
print(sentence)
print(csv_line)
```

Output: *Python is readable / red, green, blue*

```
numbers = [4, 8, 12]
text = ", ".join(str(n) for n in numbers)
print(text)
```

Output: *4, 8, 12*

zip(): pair items from iterables

```
names = ["Asha", "Ben", "Chen"]
scores = [88, 91, 79]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

`zip()` stops at the shortest input. Python 3.10+ supports `zip(names, scores, strict=True)` to raise `ValueError` if lengths differ.

```
keys = ["name", "level"]
values = ["Mina", 2]
profile = dict(zip(keys, values))
print(profile)
```

Output: *{'name': 'Mina', 'level': 2}*

slice(): save a slicing rule

A slice has the form `sequence[start:stop:step]`. `slice()` creates a reusable slice object.

```
text = "abcdefgh"
middle = slice(2, 6)
every_second = slice(None, None, 2)
print(text[middle])
print(text[every_second])
```

Output: *cdef / aceg*

Start is included, stop excluded, and a negative step moves backward.

16. Swap variables and merge dictionaries

Swap with tuple unpacking

```
left = "tea"
right = "coffee"
left, right = right, left
print(left, right)
```

Output: *coffee tea*

Python evaluates the right-hand values before assigning the left-hand names, allowing a clear swap without a temporary variable.

Merge dictionaries

When dictionaries share a key, these merge forms use the right-hand value for that key.

```
defaults = {"theme": "light", "font_size": 12}
settings = {"theme": "dark", "sound": True}

# Python 3.9+
combined = defaults | settings
print(combined)
```

Output: *{'theme': 'dark', 'font_size': 12, 'sound': True}*

For earlier Python 3 versions, dictionary unpacking works:

```
combined = {**defaults, **settings}
```

The in-place form `defaults |= settings` updates the left dictionary.

> **Check collisions**
> If duplicate keys mean data loss in your program, detect collisions before merging.

17. Find the most frequent value

The standard library's `collections.Counter` counts hashable values such as strings, numbers, and tuples.

```
from collections import Counter

colors = ["blue", "red", "blue", "green", "blue", "red"]
counts = Counter(colors)
print(counts)
print(counts.most_common(1))
```

Output: *Counter({'blue': 3, 'red': 2, 'green': 1}) / [('blue', 3)]*

`most_common(1)` returns a list with at most one (value, count) pair. Empty input returns an empty list, so check before reading its first item.

Handle ties explicitly

```
from collections import Counter

values = ["red", "blue", "red", "blue"]
counts = Counter(values)
highest = max(counts.values())
first_seen = dict.fromkeys(values)
winners = [v for v in first_seen if counts[v] == highest]
print(winners, highest)
```

Output: *['red', 'blue'] 2*

There can be several most-frequent values. Decide whether to return one winner, every tied winner, or report a tie.

18. Count the digits in an integer

Count decimal digits without counting a minus sign. Zero has one digit. A simple approach measures the length of the absolute value as text.

```
number = int(input("Enter a whole number: "))
digit_count = len(str(abs(number)))
print(f"It has {digit_count} digit(s).")
```

`abs(-405)` is `405`; its text has three characters.

Count with arithmetic

```
def count_digits(number):
    """Count base-10 digits; zero has one digit."""
    number = abs(number)
    if number == 0:
        return 1
    count = 0
    while number > 0:
        number //= 10
        count += 1
    return count

for sample in [0, 7, 405, -12000]:
    print(sample, count_digits(sample))
```

Output: *0 1 / 7 1 / 405 3 / -12000 5*

Each floor division by 10 removes the last decimal digit. The loop counts how many times this happens before the number reaches zero. Decimal values need a separate rule for which digits count.

Practice: try these before reading the answers

Write a short solution for each. Predict behavior first, then run with at least two inputs.

- Ask for a name; if the user enters only spaces, ask for a name.
- Ask for two numbers and print sum, difference, product, and quotient. Handle a zero divisor.
- Given scores, create a list containing only scores of at least 50, doubled.
- Write a function that returns the largest value in a list. Decide what empty input means.
- Read a file line by line and count its non-empty lines. Handle a missing file.
- Number a list of words starting at 1, then join them into a sentence.
- Find every most-frequent value in a list.
- Ask until the user enters an integer from 1 through 10.

Sample answers

These are possible approaches; other correct solutions exist.

```
name = input("Name: ").strip()
if name:
    print(f"Hello, {name}!")
else:
    print("Please enter a name.")

scores = [35, 50, 72, 48]
passed_doubled = [s * 2 for s in scores if s >= 50]

words = ["read", "run", "change"]
for number, word in enumerate(words, start=1):
    print(f"{number}. {word}")
sentence = " ".join(words)
```

> **Keep practicing**
> When code fails, reduce it to the smallest example that still shows the issue. Check types, empty input, and collection lengths. Correct one assumption at a time.

Quick reference

- Display a value: `print(value)`.
- Read input: `text = input(prompt)`.
- Convert text: `number = int(text)`.
- Choose a path: `if / elif / else`.
- Repeat over items: `for item in iterable:`.
- Repeat until a condition changes: `while condition:`.
- Create reusable behavior: `def name(parameters):`.
- Return a result: `return value`.
- Build a transformed list: `[expression for item in items]`.
- Open text safely: `with open(path, mode, encoding="utf-8") as file:`.
- Handle a known failure: `try / except SpecificError`.

Core habits

- Use names that explain purpose.
- Choose collections by the operations and data they support.
- Convert input deliberately and handle expected invalid input.
- Keep functions focused and use return values to share results.
- Use `with` for files and choose file modes carefully.
- Prefer readable code over clever code, especially while learning.

Values flow through variables, operators transform them, conditions and loops control execution, collections organize data, functions package logic, exceptions handle expected failures, and files preserve results.